

Lab session 5

Near-duplicate detection using LSH

Cerca i Anàlisi de la Informació (CAI)

David González

Angel Morales

November 2025

Table of contents

Introduction	3
Exercise 1.	4
Exercise 2.	5
Exercise 3.	6
Results	9
Conclusions	12

Introduction

In this report we study how locality-sensitive hashing (LSH) is used to find near-duplicate documents. We see how the parameters m , k optimize the algorithm to be able to properly distinguish documents. To evaluate the implementation of LSH, we have used cosine similarity.

The main experiment has consisted on setting some pairs for the possible m and k parameters, then computing the has tables for a corpus provided by the teacher for every pair of m and k , checking which document pairs fall on the same bucket, and checking if the cosine similarity is over an arbitrary threshold (we have used 0.9). If the pair's cosine similarity is equal to or greater than the threshold, we count this pair as a true positive. Otherwise, we count it as a false positive. This allows us to measure the accuracy of our system.

Prior to the experiment, we have done some exercises proposed by the teacher to ensure that we understand how LSH works.

Exercise 1.

We will show a toy example of how the simhash of a document d is computed. We will suppose we have the document formatted as a dictionary, where the keys are the (unique) terms inside the document and the values are the tf-idf weights.

$$d = \{\text{"bit": 0.4, "coin": 1.2}\}$$

The first thing to do would be to compute a random hash, in the case we will use a provided md5 hash of 4 bits, and we will suppose it is random.

$$h(\text{"bit"}) = (1, 1, 1, 1)$$

$$h(\text{"coin"}) = (1, 0, 0, 1)$$

Now, to compute the document hash, we have to sum the hashes multiplied by its tf-idf weight. But, prior to that, 0s should become -1s.

$$h'(\text{"bit"}) = (1, 1, 1, 1)$$

$$h'(\text{"coin"}) = (1, -1, -1, 1)$$

$$\begin{aligned} h(d) &= h'(\text{"bit"}) \cdot 0.4 + h'(\text{"coin"}) \cdot 1.2 = (0.4 + 1.2, 0.4 - 1.2, 0.4 - 1.2, 0.4 + 1.2) = \\ &= (0.4 + 1.2, 0.4 - 1.2, 0.4 - 1.2, 0.4 + 1.2) = (1.6, -0.8, -0.8, 1.6) \end{aligned}$$

Lastly, we map positives to 1 and negatives to 0.

$$h(d) = (1, 0, 0, 1)$$

The binary simhash for the document is 1001. Although k and m parameters are not needed to compute the simhash, they are provided for this exercise: $k = 4$, $m = 1$. What we can do is check that the number of bits of the hash can be used with those parameters. We see that $k \cdot m = 4$, which is equal to the number of bits of 1001, so we can use these parameters without any problem.

Exercise 2.

The teacher has provided us with a template to build the code which, given a document, returns its simhash, just the way we did for the toy example in exercise 1. She has also given us some auxiliary functions to compute hashes and load the document to perform the tests.

Although all the code can be found in the delivered Python Notebook, this is the code completed for this exercise:

```
# Exercise 2: function to compute the simhash

## constants
K = 18
M = 3
B = M*K

def _simhash(id: str, b: int) -> list[int]:
    """
    id is the document id, b is the desired length of the simhash
    it should return a bitstring of length b as explained in the
    first section of the notebook
    """

    # Access document
    words_weights: dict[str, np.float] = (corpus[id]) # dict[word: tf-idf]
    #print(words_weights) # debug
    bitstrings: list[list[int]] = []
    # For each word, compute the _termhash(w, b)
    for word, weight in words_weights.items():
        bitstring = _termhash(word, b)

        # Replace 0 with -1
        new_bitstring = []
        for char in bitstring:
            if char == "1": new_bitstring.append(weight)
            elif char == "0": new_bitstring.append(-weight)
            else: raise ValueError("One bit is not 0 neither 1.")
        assert len(new_bitstring) == b

        bitstrings.append(new_bitstring)

    # Follow instructions to compute the simhash of the document (round to +-1, sum, round to 1, 0)
    doc_hash = [sum(bitstrings[w][bit] for w in range(len(bitstrings))) for bit in range(b)]
    assert len(doc_hash) == b

    # Replace positives with 1 and negatives with 0
    new_doc_hash = []
    for item in doc_hash:
        if item > 0: new_doc_hash.append(1)
        else: new_doc_hash.append(0)
    assert len(new_doc_hash) == b

    return new_doc_hash
```

✓ 0.0s Python

Figure 1: code for exercise 2 - document simhash

It basically does what we did for exercise 1: compute each word's hash, replace zeros with minus ones, perform the dot product, and map positives to 1 and negatives to 0.

Exercise 3.

This is the important part of the report. It basically consists of studying the effect of the k and m parameters of the LSH on a practical scenario. The Python Notebook contains the details on the code developed, which allows us to distribute the document's hashes in buckets, check for collisions (a "collision" is one pair of documents which share at least one bucket), and use the cosine similarity to classify each collision as a true positive (TP) and false positive (FP). This classification is done by, for each collided pair, checking if the cosine similarity between the documents is equal to or higher than an arbitrary threshold, which we have set to 0.9 because we intend to detect near duplicate documents. Pairs with similarity ≥ 0.9 have been classified as TP, the rest as FP.

With that code, we have been able to define some experiments which consist on giving a pair (m, k) as an input, computing the hash table, checking the collisions, classifying them, and returning the TP, FP and some other metrics: accuracy, time taken to build the hash table (in seconds) and memory size of the hash table (in GBs). A computer with 32GBs of RAM and an Intel Core i5-12600K with a base clock of 3.69 GHz and 16 threads has been used to perform all computations.

We expected to see that an increase in k is related to higher accuracy, and an increase in m is related to a higher recall. But how exactly? And what about the resources needed?

We initially set some diverse values for (m, k) , centered around the teacher's given $(3, 18)$. Results can be seen at Table 1.

	m	k	Hits	TP	FP	Accuracy	Hash computation time	Hash size (GBs)
0	1	10	0	0	0	1.000000	42.551983	0.003568
1	1	18	0	0	0	1.000000	50.105449	0.020119
2	1	23	0	0	0	1.000000	63.480272	0.507493
3	3	10	9179	118	9061	0.012855	63.206190	0.004644
4	3	18	32706	13463	19243	0.411637	85.000998	0.054291
5	3	23	13943	13177	766	0.945062	129.108942	1.516418
6	5	10	50893	390	50503	0.007663	80.829140	0.005721
7	5	18	54051	16356	37695	0.302603	118.631979	0.088483
8	5	23	17217	15724	1493	0.913283	188.599383	2.525341

Table 1: results for the first input set

We have plotted the accuracy in a heatmap (Figure 2).

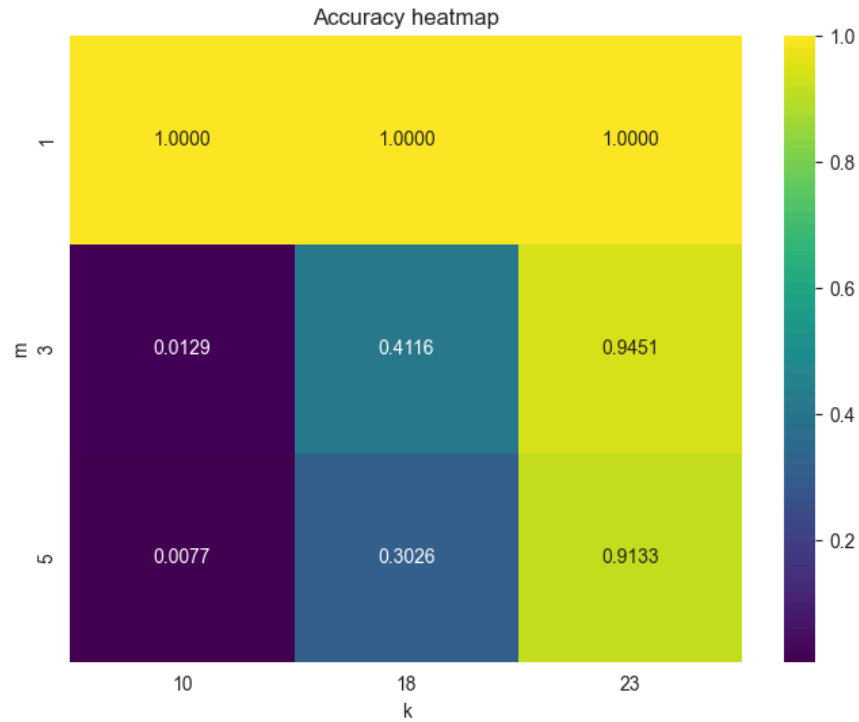


Figure 2: Accuracy heatmap for some values of m and k

After trying with some various combinations of parameters, we see that that:

- Low k causes almost all collisions to be false positives.
- Low m causes collisions to be too unlikely, and we can get cases with zero hits.
- Combinations (3, 25) and (5, 25) had the best accuracies, although they have the highest computation time.

We can now try to get a higher accuracy, but we may sacrifice the computation time and need much more memory. The best result until now was $m = 3$ and $k = 23$, with an accuracy of 0.945.

	m	k	Hits	TP	FP	Accuracy	Hash computation time	Hash size (GBs)
0	2	24	10193	9990	203	0.980084	116.228583	2.021664
1	2	25	10062	9956	106	0.989465	162.006749	4.044749
2	3	24	13542	13104	438	0.967656	161.656059	3.030981
3	3	25	13290	13055	235	0.982318	230.215361	6.065606
4	4	24	15586	14957	629	0.959643	198.857278	4.040297
5	4	25	15251	14890	361	0.976329	289.158406	8.086463

Table 2: results for the second input set

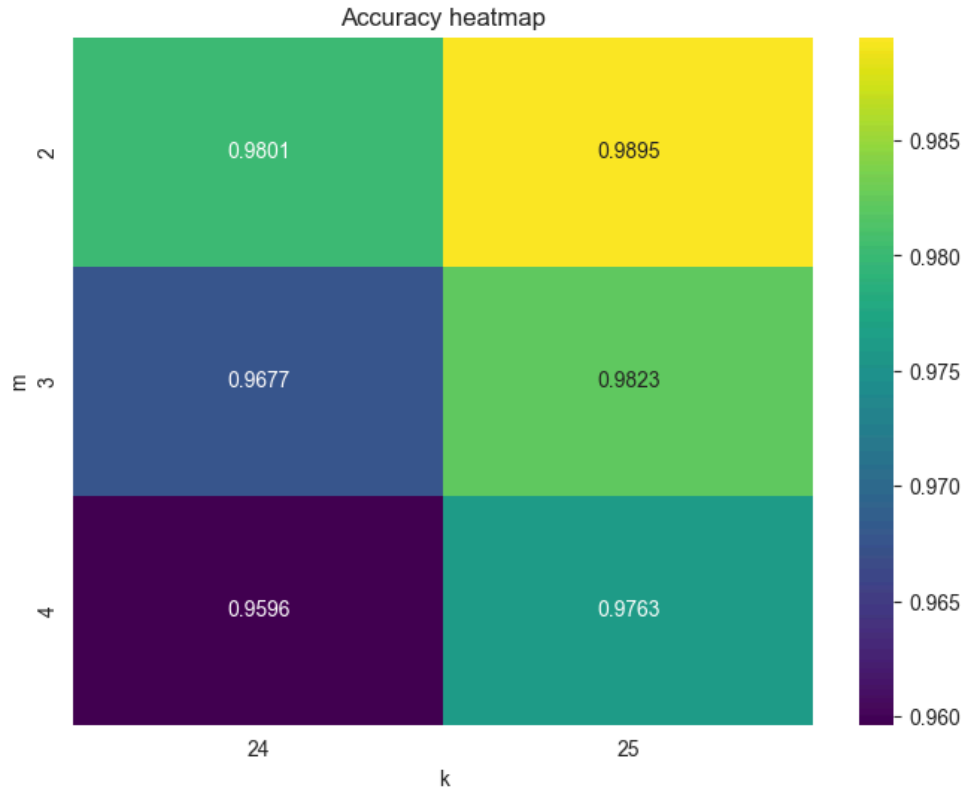


Figure 3: Accuracy heatmap for high values of k

We see that increasing k and lowering m seems the combination with the highest accuracy. We can try to get even better results, but we have to be careful because the memory usage is exploding, and we already managed to occupy 8GBs for the hash table with $m = 3$ and $k = 25$ (see Table 2).

	m	k	Hits	TP	FP	Accuracy	Hash computation time	Hash size (GBs)
0	2	26	9983	9929	54	0.994591	371.946769	8.098360
1	2	27	9923	9888	35	0.996473	483.527860	16.220685

Table 3: results for the third input set

Increasing k from 26 to 27 causes an increase of the accuracy of a 0.002, relatively imperceptible, and doubles the memory usage from 8 to 16 GBs. We are being clearly too greedy.

Results

Lets see our results and discuss which are the best combinations. Graphics can be seen at a higher resolution in the Python Notebook.

	m	k	Hits	TP	FP	Accuracy	Hash computation time	Hash size (GBs)
0	1	10	0	0	0	1.000000	42.551983	0.003568
1	1	18	0	0	0	1.000000	50.105449	0.020119
2	1	23	0	0	0	1.000000	63.480272	0.507493
3	3	10	9179	118	9061	0.012855	63.206190	0.004644
4	3	18	32706	13463	19243	0.411637	85.000998	0.054291
5	3	23	13943	13177	766	0.945062	129.108942	1.516418
6	5	10	50893	390	50503	0.007663	80.829140	0.005721
7	5	18	54051	16356	37695	0.302603	118.631979	0.088483
8	5	23	17217	15724	1493	0.913283	188.599383	2.525341
0	2	24	10193	9990	203	0.980084	116.228583	2.021664
1	2	25	10062	9956	106	0.989465	162.006749	4.044749
2	3	24	13542	13104	438	0.967656	161.656059	3.030981
3	3	25	13290	13055	235	0.982318	230.215361	6.065606
4	4	24	15586	14957	629	0.959643	198.857278	4.040297
5	4	25	15251	14890	361	0.976329	289.158406	8.086463
0	2	26	9983	9929	54	0.994591	371.946769	8.098360
1	2	27	9923	9888	35	0.996473	483.527860	16.220685

Table 4: all results

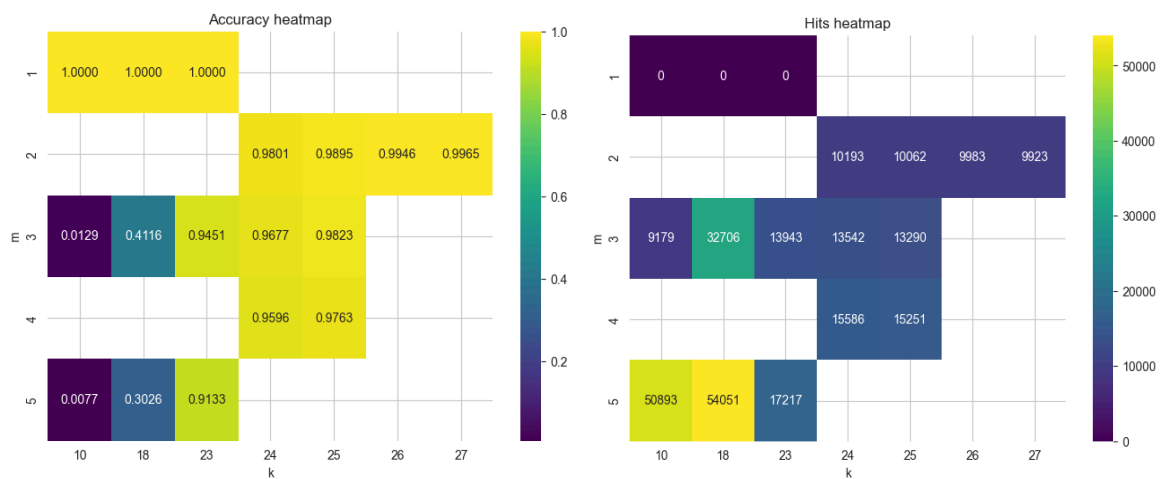


Figure 4: accuracy (left) and hits (right) heatmaps (independent color scales)

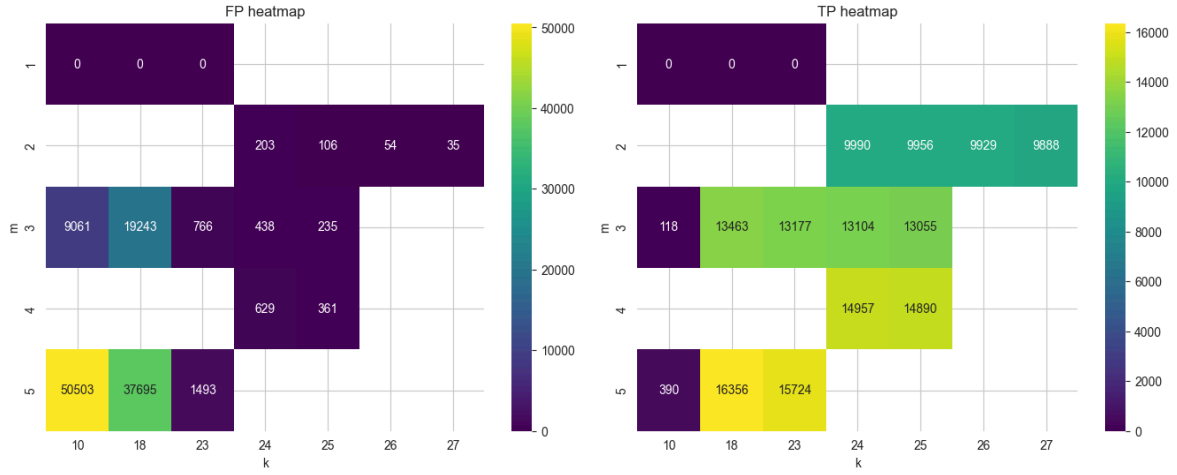


Figure 5: FP (left) and TP (right) heatmaps (independent color scales)

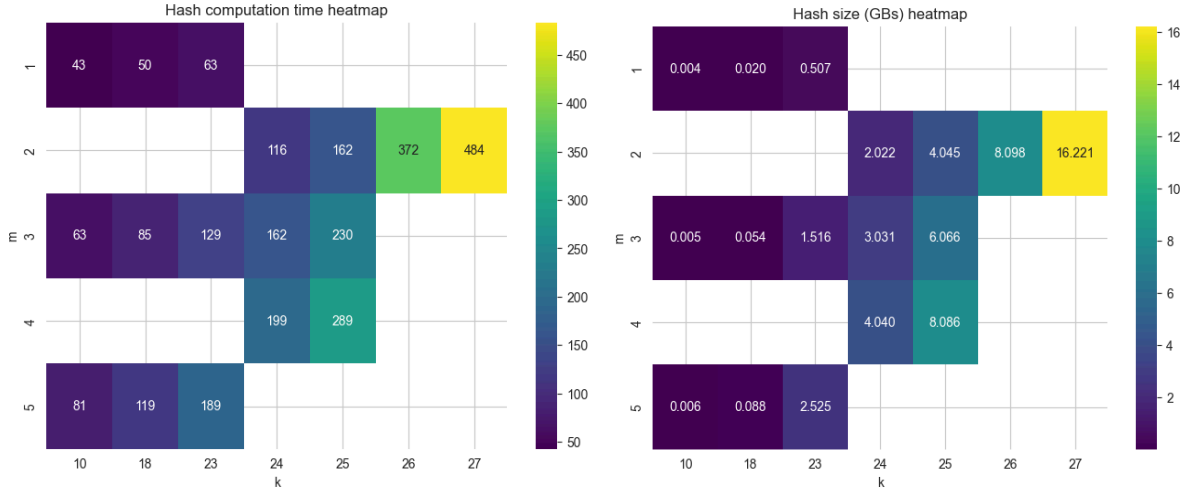


Figure 6: Hash computation time (s) (left) and hash size (GBs) (right) heatmaps (independent color scales)

In terms of accuracy, we start to see acceptable rates for $k = 23$ (probably k can be a bit smaller), and we need m to be at least 2. In fact, m should not be greater than 2 to avoid losing accuracy. If, instead, we wanted to maximize recall (we will later talk about this metric), we may want to increase m .

If our system is capable of handling between 2 and 6 GBs, which is something usual nowadays that most systems have 16GBs, we can opt for a k around 24 or 25, and m around 2 or 3. This will give us a pretty good accuracy around 0.97. If we want to achieve high accuracy with lower resource consumption, k should be 23 and m should be 2. Note that this only relates to personal computers, not to distributed systems or high performance computing. Nevertheless, the scalability behaves the same.

If we have spare memory (32GBs or more), we can set k to 26 and m to 2 or probably even 3. We could even set k to 27, but it is not worth doubling the RAM usage to have an increase of around a 0.002%.

For every case, we may expect the computation time to be around 2 or 3 minutes, larger times if our system has a modest processor.

What about recall? Recall can only be computed if we know the true number of duplicates in our corpus. Doing this by brute force would imply computing the cosine similarity of each pair of documents, which means computing the 60 thousand norms, the 120 million cosine similarities, and the thousands of millions of products needed to do that. We can't even estimate how long it would take.

What we can actually estimate is an upper bound for the recall. We can see that for $m = 5$, $k = 18$, there are 16.356 true positives, which means that the number of positives inside the corpus is at least this big. We can divide the number of true positives of every case by this number to get an upper bound for the recall on each one.

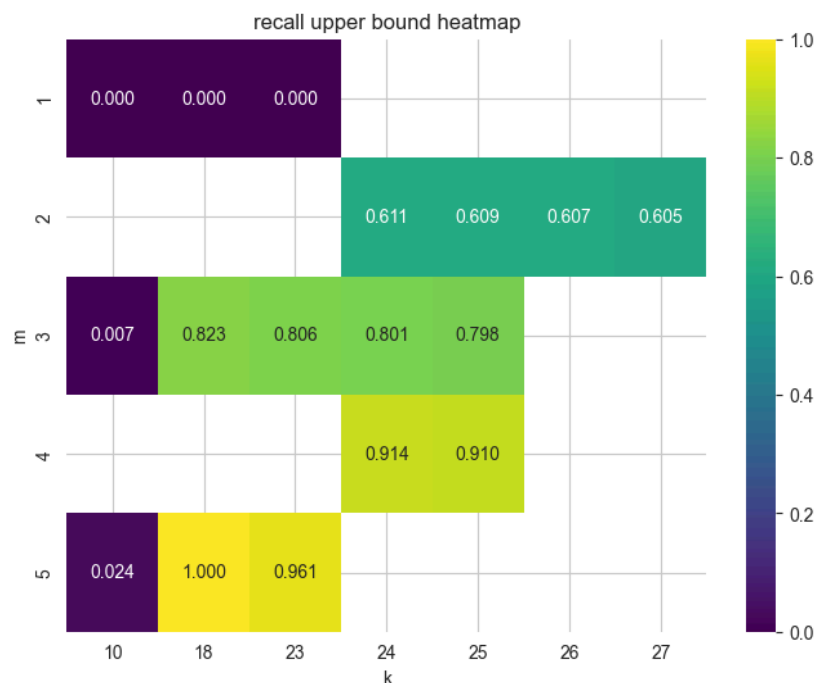


Figure 7: Recall upper bound heatmap

And now we finally see how the balance between accuracy and recall is present in a practical case. Increasing both is much more expensive than sacrificing one of them in exchange for the other.

Conclusions

We have tested the balance between accuracy and recall, and effectively checked that the cost of having them both may be too high to handle, and not scalable.

- "To improve accuracy, increase k ."
- "To improve recall, increase m ."
- Of course, be careful not to increase them too much, or be ready to have your resources consumed.

We have also seen how choosing m and k parameters for LSH is not trivial, and is a very important task to ensure the right spending of the available resources.

We have also seen the utility of locality sensitive hashing (LSH) to detect near-duplicate documents over a desired cosine similarity threshold, which is much more efficient than using brute force.

The highest accuracy achieved has been of a 0.9965, which corresponds to 35 false positives out of almost 10,000 hits. We have also estimated an upper bound for the recall and discovered how this accuracy sacrifices the recall, which cannot be greater than 0.6 (and may be quite lower than that).